



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 03

Team No.:07

Project Title: Linux Process Resource Monitoring System

Team Members

Roll Number	Student Name	Signature
2520030010	G.Boomika Sre	
2520030174	P.Nehasree	
2520030282	P.Hasini	

1. Abstract

Linux systems run multiple processes at the same time. Sometimes a process may consume more CPU time or memory than expected, remain active for a long period, or affect the performance of other programs. Therefore, users need a simple way to observe running processes and understand their basic behavior.

This project proposes a command-line **Linux Process Resource Monitoring System** implemented in C. The system will display basic information about currently running processes, such as Process ID (PID), process name, process state, and parent process ID. It will read available process-related information from the Linux /proc directory and show the details in a clear terminal-based format. The program will also provide simple options to refresh the process list and send basic signals, such as SIGTERM, to a user-selected permitted process.

The project will use fundamental Linux and POSIX concepts, including processes, process IDs, parent-child relationships, signals, directory handling, file input/output, and error handling. Relevant functions may include opendir(), readdir(), open(), read(), close(), getpid(), fork(), wait(), and kill().

The expected outcome is a simple working Linux user-space tool that helps users view and manage basic process information. The project will strengthen understanding of process creation, process monitoring, signals, and Linux system-programming interfaces.

2. Problem Statement

Linux operating systems run multiple processes simultaneously, and each process may use system resources such as CPU time and memory. When many processes are active, it can become difficult for users to identify running processes,

understand their current state, and monitor their basic resource usage. A process that consumes excessive resources or remains active unnecessarily may affect overall system performance.

The project aims to develop a Linux-based Process Resource Monitoring System in C that provides users with a simple command-line interface to view and monitor currently running processes. The system will collect process-related information from the Linux /proc filesystem and display details such as Process ID (PID), process name, process state, parent process ID, CPU usage, and memory-related information.

From an Operating Systems and Systems Programming perspective, the project involves process management, process identification, parent-child relationships, Linux filesystem handling, file I/O, directory operations, signal handling, and system calls. The system will also provide basic process-management functionality by allowing permitted processes to be selected and signals such as SIGTERM to be sent.

The project addresses the challenge of providing a simple and understandable user-space tool for observing process activity and basic resource utilization in a Linux environment.

3. Objectives

1. **To develop** a Linux-based command-line process monitoring system in C for displaying information about currently running processes.
2. **To read and analyze** process information from the Linux /proc filesystem and display details such as PID, process name, process state, parent PID, CPU usage, and memory usage.
3. **To implement** Linux and POSIX system-programming concepts such as directory handling, file I/O, process identification, process creation, waiting, and signal handling.
4. **To provide** basic process-management features such as refreshing the process list and sending permitted signals such as SIGTERM to a selected process.

4. Proposed Methodology

The project will be implemented in **Linux using C** as a command-line based process monitoring application. The system will access the /proc virtual filesystem to obtain information about currently running processes.

The program will identify directories corresponding to process IDs and read relevant files inside each process directory to obtain process details. The collected information will be processed and displayed in a structured terminal format. Users will be able to refresh the process list to observe updated process information and select permitted processes for basic signal operations.

The system will use Linux/POSIX functions such as opendir(), readdir(), open(), read(), close(), getpid(), fork(), wait(), and kill() wherever required. Appropriate error handling and permission checks will be used while accessing process information or sending signals.

Project Flow:

User → Process Monitor → Filesystem → Read Process Information → Process Data → Display in Terminal → User Action / Signal

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
Process Management	Identifies and monitors currently running Linux processes.
Process ID (PID)	Uniquely identifies individual processes.
Parent-Child Process Relationship	Displays the relationship between a process and its parent process.
/proc Filesystem	Provides process-related information such as process state, CPU, and memory details.
Directory Handling	Uses opendir() and readdir() to locate and read process directories.
File I/O System Calls	Uses open(), read(), and close() to access process information from /proc.
Signals	Provides communication and basic control between the monitoring system and processes.
kill() System Call	Sends permitted signals such as SIGTERM to a selected process.
getpid()	Obtains the PID of the currently running monitoring process.
fork()	Creates a child process when process creation is required.
Error Handling	Handles invalid process entries, inaccessible files, and permission-related errors.

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030010	G. Boomika Sre	Process Identification & /proc Handling: Implement process discovery, PID identification, /proc directory traversal using opendir() and readdir(), and extraction of basic process information.
2520030174	P. Nehasree	Resource Monitoring & Display: Implement collection and processing of CPU and memory-related information, process state, parent PID, and structured command-line display and refresh functionality.
2520030282	P. Hasini	Process Management & System Calls: Implement signal handling, permitted process control using kill(), process creation/wait functionality where required, error handling, and integration/testing of the system.

7. Tools / Platforms / Software Used

(List the programming languages, operating system, libraries, tools, and software planned for the project.)

Tool / Platform / Software	Purpose
Linux / Ubuntu	Provides the operating-system environment for developing and running the process monitoring system.

C / C++	Used to implement process monitoring, /proc handling, system calls, signals, and file operations
GCC	Compiles and builds the C source code.
POSIX APIs	Provides standard interfaces for directory handling, process management, signals, and system-level operations.
Linux /proc Filesystem	Provides process and system information exposed by the Linux kernel.
Linux System Calls / APIs	Used for process management, file I/O, signals, and other system-level operations.
Git / GitHub	Used for source-code version control and project collaboration.

8. Expected Outcome

The expected outcome is a functional Linux command-line application that displays information about currently running processes in a clear and organized format. The application will show fields such as PID, process name, process state, parent process ID, CPU usage, and memory usage.

Users will be able to refresh the process list, search for a process by PID or name, view detailed process information, and send a safe termination signal to a permitted process. The system will validate inputs and provide meaningful error messages for invalid PIDs, unavailable processes, inaccessible files, and insufficient permissions.

The project will provide practical experience with Linux process management, the /proc filesystem, directory handling, file I/O, signals, process creation, synchronization, and POSIX system-programming interfaces. It will result in a lightweight educational alternative for understanding how process-monitoring utilities work internally.

9. GitHub Repository

GitHub Repository Link: https://github.com/nehasree2507-hash/Linux-Process-Resource-Monitoring-System_OSSP

Repository Created: Yes

☐ Yes ☐ No

Faculty Name and Signature

Faculty Name: K. Hema

Signature: _____

Remarks: _____

Faculty Use

Project Approved: _____

☐ Yes ☐ No

Date: _____

Faculty Signature: _____